Abstract

Developing reliable data pipelines traditionally demands deep engineering expertise in distributed systems, workflow orchestration, and data transformation framework creating a significant barrier for domain experts who understand the data but lack programming proficiency. This paper presents AutoPipeline, a comprehensive, configuration driven framework that bridges this gap by transforming plain English descriptions into fully executable Apache Airflow DAGs. The system automates three core engineering tasks that normally require hours of manual work: (1) intelligent schema detection that automatically infers data types and quality metrics from raw data across CSV, JSON, Parquet, REST API, and Apache Kafka sources; (2) AI-powered transformation generation that uses Google Gemini to produce validated, sandboxed PySpark code directly from natural language prompts; and (3) template-based DAG orchestration that reliably generates Apache Airflow pipelines without any manual configuration. The framework implements the Medallion Architecture (Bronze,Silver,Gold layers) on MinIO object storage and is deployed as 13 microservices via Docker Compose. Through three real-world case studies CSV file uploads, Apache Kafka IoT streams, and REST API ingestion we demonstrate schema detection confidence scores of 0.88–0.95, syntactically correct code generation in all test scenarios, and a reduction in end-to-end pipeline development time from 8 to 10 hours to approximately 2 minutes (a 99.6% reduction).

Keywords: Data pipelines, Workflow automation, Large Language Models, Apache Airflow, Medallion Architecture, Schema detection, PySpark, Google Gemini, Apache Kafka, Microservices

1. Introduction

Data pipelines are the backbone of modern data-driven organisations. They orchestrate the systematic collection, processing, transformation, and delivery of information across diverse systems from operational databases to analytics dashboards and machine-learning models. Without well-engineered pipelines, raw data remains locked in silos, inaccessible for the business decisions that depend on it. Yet building these pipelines has always been the exclusive domain of specialised data engineers. Frameworks such as Apache Airflow are extremely powerful, but they demand a working knowledge of Python, DAG semantics, operator configuration, Spark job management, and infrastructure provisioning. A domain expert, say, a supply-chain analyst who knows exactly which transformations their data require is typically unable to translate that knowledge directly into executable pipelines without engineering support. This skills gap slows down innovation and creates organisational bottlenecks. The rise of Large Language Models (LLMs) offers a compelling path forward. Modern LLMs demonstrate remarkable ability to understand intent expressed in natural language and to generate syntactically correct, semantically meaningful code. However, applying LLMs naively to pipeline generation asking a model to produce an entire Airflow DAG from a single prompt tends to yield fragile, inconsistent results. The DAG structure must be deterministically correct; LLM creativity is a liability here, not an asset. AutoPipeline resolves this tension with a hybrid strategy: LLM

flexibility is applied precisely where it adds the most value (transformation logic, where natural language intent must be faithfully translated into PySpark operations), while DAG orchestration is generated deterministically from battle-tested Jinja2 templates (where correctness is non-negotiable). The result is a system that is simultaneously accessible to non-engineers and reliable enough for production use.

1.1 Problem Definition The fundamental challenge AutoPipeline addresses can be stated as:

How can we transform data pipeline development from a code-centric, engineering-intensive process into a configuration-driven, AI assisted workflow that democratises access to data engineering capabilities? Concretely, we automate the following three tasks that normally require manual effort:

• Schema Detection: Automatically infer column data types, nullability, and data quality metrics from raw data whether files, REST API responses, or live Kafka stream without requiring the user to write a single line of schema-definition code.

• Transformation Logic: Convert plain English descriptions of desired data transformations (e.g., "filter cancelled flights and categorise delays into four buckets") into production-ready PySpark functions, complete with validation, dry-run testing, and iterative refinement through conversation.

• Pipeline Orchestration: Automatically generate Apache Airflow DAGs that wire together ingestion, transformation, and loading stage including retry policies, scheduling, and dependency chain without the user ever needing to write orchestration code.

1.2 Key Contributions

• A modular 13-service microservices architecture integrating AI code generation with Apache Spark, Apache Airflow, MinIO, PostgreSQL, and Apache Kafka in a single Docker Compose deployment.

• A prompting methodology for LLM-based PySpark code generation that injects live schema context, maintains full conversation history, and validates generated code in a sandboxed environment before any real data is touched.

• Template-based DAG generation using Jinja2 that eliminates manual Airflow configuration while preserving full flexibility for complex multi-stage workflows.

• Medallion Architecture implementation on MinIO providing immutable Bronze, validated Silver, and business-ready Gold data layers with complete lineage tracking.

• Experimental validation through three end-to-end case studies demonstrating schema detection confidence of 0.88-0.95 and 99.6% reduction in pipeline development time.

2. Background and Related Work

2.1 Data Pipeline Orchestration Frameworks

Modern pipeline orchestration has matured into a rich ecosystem. Apache Airflow [1] has emerged as the de-facto open-source standard, representing workflows as Directed Acyclic Graphs (DAGs) authored in Python. Its extensible operator library, rich UI, and active community make it extremely capable, but also demanding to operate correctly. Other notable frameworks include Luigi (Spotify's dependency-focused solution), Prefect [3] (emphasising dynamic

workflows and automatic retries), Dagster [4] (data-aware orchestration with first-class asset tracking), and cloud-native options like AWS Step Functions [8]. Despite their power, all these frameworks share a common limitation: they require users to possess deep programming knowledge, which excludes the majority of domain experts from directly authoring pipelines.

2.2 LLM-Based Code Generation

Large Language Models trained on code such as OpenAI Codex [9], Code Llama [10], and StarCoder [12] have demonstrated impressive capabilities in generating syntactically correct code from natural language prompts. However, research consistently shows that end-to-end generation of complex multi-component systems is unreliable [11, 12]. Generated code may appear plausible in isolation yet fail when integrated into a larger system due to incorrect assumptions about the runtime environment, missing imports, or subtle semantic errors. Prompt2DAG [13] provides the closest prior work to AutoPipeline, systematically comparing four generation strategies Direct, LLM-only, Hybrid, and Template-based for producing data enrichment pipelines. Their key finding is that hybrid approaches achieve the best balance of flexibility and reliability, a conclusion that directly informs AutoPipeline's design philosophy.

2.3 Medallion Architecture

The Medallion Architecture, popularised by Databricks [14], organises a data lake into three progressively refined layers. The Bronze layer is an immutable landing zone preserving raw data exactly as ingested. The Silver layer contains validated, deduplicated, and type-corrected data. The Gold layer holds business-ready aggregated datasets optimised for analytics and machine learning. This layered approach provides clear data lineage, supports incremental reprocessing, and enables easy rollback when data quality issues are discovered.

3. Methodology

3.1 System Architecture Overview

AutoPipeline is designed as a modular, microservices-based system separating concerns across three major functional components: (i) Multi-Source Data Ingestion, (ii) Agentic Data Transformation, and (iii) Pipeline Orchestration. All 13 services are deployed together via a single docker compose up command, making the entire stack reproducible on any machine with Docker installed. The architectural philosophy is configuration-driven: users interact through a React-based frontend, which communicates with a FastAPI backend that coordinates all pipeline operations. Apache Spark handles distributed data processing; MinIO provides S3-compatible object storage implementing the Medallion Architecture; Apache Kafka enables real-time streaming ingestion; PostgreSQL stores metadata and exports final Gold data; and Apache Airflow orchestrates end-to-end workflow execution. Google Gemini serves as the underlying LLM for transformation code generation.

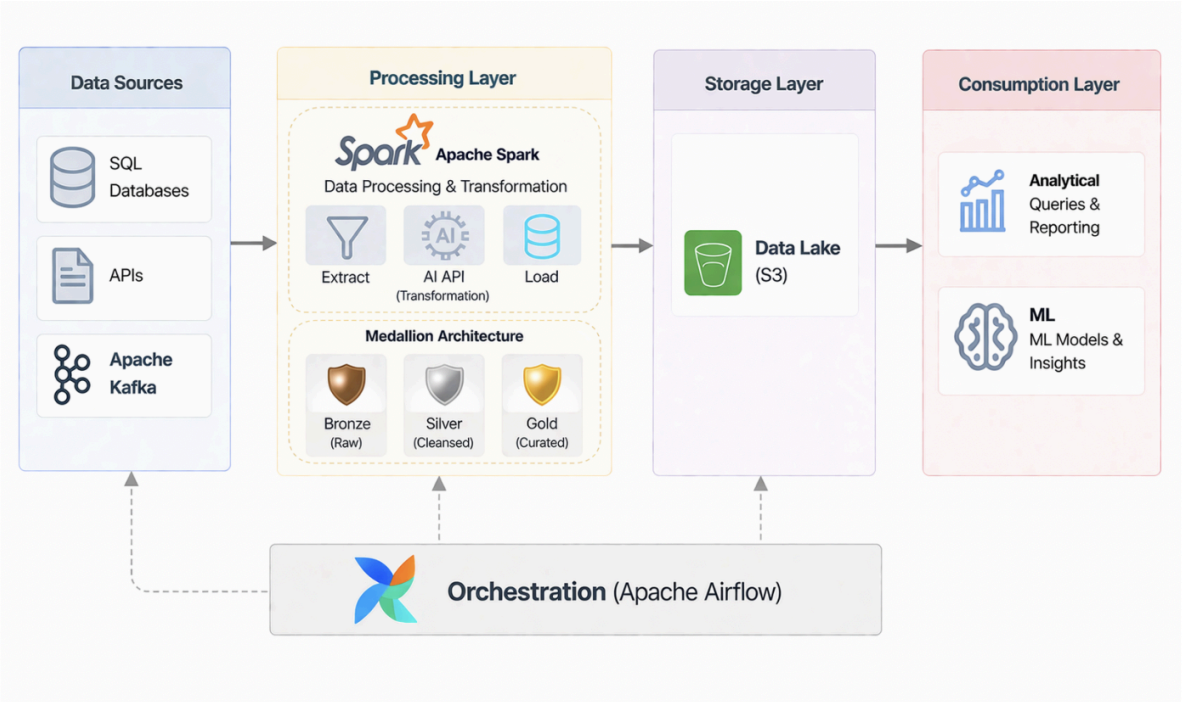

Figure 1: AutoPipeline System Architecture — Data sources (SQL databases, APIs, Apache Kafka) feed into a Processing Layer (Apache Spark with Extract–AI Transform–Load pipeline and Medallion Architecture), stored in a Data Lake (S3/MinIO), and consumed by Analytical and ML workloads. Apache Airflow orchestrates all layers end-to-end.

Table 1 lists every service in the AutoPipeline stack:

Service	Technology	Role
PostgreSQL	15-alpine	Metadata storage; Gold data export target
MinIO	Latest	S3-compatible object storage (Bronze / Silver / Gold buckets)
Redis	7-alpine	Response caching for repeated API calls
Zookeeper	CP 7.6.0	Kafka cluster coordination
Kafka	CP 7.6.0	Real-time streaming data source
Spark Master	3.5.1	Distributed processing coordination
Spark Worker	3.5.1	Processing execution node
FastAPI Backend	0.109.0	REST API gateway; all business logic
Airflow Webserver	2.9.0	DAG monitoring and management UI
Airflow Scheduler	2.9.0	DAG execution scheduling and triggering
React Frontend	18.2	User-facing web interface
Google Gemini	2.5-flash	LLM for PySpark transformation code generation

Table 1: AutoPipeline Service Architecture (13 microservices)

3.2 Medallion Architecture Implementation

AutoPipeline stores all data in three purpose-built MinIO buckets that map directly to the three Medallion layers. Each layer has clearly defined responsibilities, ensuring that data at every stage is traceable, reprocessable, and independently consumable.

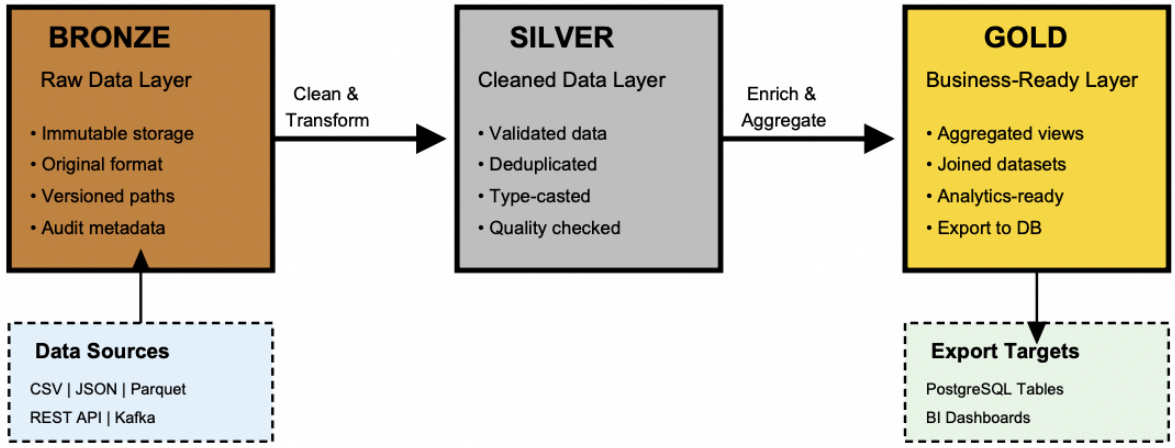

Figure 2: *Medallion Architecture data flow showing Bronze (raw), Silver (cleaned), and Gold (enriched) layers with transformation stages.*

Layer	MinIO Bucket	Purpose & Characteristics
■ Bronze (Raw)	`s3a://bronze/{project}/v{n}/`	Immutable landing zone. Data stored exactly as received, partitioned by ingestion date. Metadata columns (_ingestion_timestamp, _pipeline_id, _schema_version) appended automatically.
■ Silver (Cleaned)	`s3a://silver/{project}/silver/`	Validated, deduplicated, type-corrected data. LLM-generated PySpark transformations applied sequentially. Output written as CSV with headers.
■ Gold (Enriched)	`s3a://gold/{project}/gold/`	Business-ready aggregated datasets. May include joins with reference tables, window functions, and aggregations. Optionally exported to PostgreSQL for BI tools.

Table 2: Medallion Architecture layer definitions

3.3 Component 1: Multi-Source Data Ingestion

The ingestion subsystem provides a unified interface for three categories of data sources. Regardless of the source type, the system automatically performs schema detection before any data is moved, giving users a chance to review and confirm the inferred structure.

3.3.1 Schema Detection Algorithm

Schema detection is the first step in every pipeline. The algorithm analyses a sample of the input data using Pandas and applies heuristic pattern matching to infer the most appropriate data type for each column. This is more robust than simply trusting Pandas' default dtype inference, which often leaves numeric-looking strings as object type. The key insight is that real-world data is noisy: a column of integers may contain a few whitespace-padded strings; a timestamp column may mix ISO and US date formats. The algorithm handles this gracefully by calculating a confidence score (0-1) for each inferred type based on the fraction of values that successfully match the pattern. The overall schema confidence combines per-field confidence with a file-compatibility ratio across all discovered files.

Algorithm 1: Schema Detection

```
Input: file_path, file_format, sample_rows = 1000
Output: SchemaDetectionResult
files ← ScanDirectory(file_path, file_format)
sample ← ReadSample(files[0], sample_rows)
schema ← EmptySchema()
for each column in sample do
base_type ← GetPandasDtype(column)
if base_type = 'object' then
if MatchesBooleanPattern(column) then
inferred_type ← BOOLEAN
else if MatchesIntegerPattern(column) then
```

```
inferred_type ← INTEGER
else if MatchesFloatPattern(column) then
inferred_type ← FLOAT
else if MatchesDatetimePattern(column) then
inferred_type ← TIMESTAMP
else if MatchesDatePattern(column) then
inferred_type ← DATE
else
inferred_type ← STRING
end if
end if
confidence ← CalculatePatternMatchRatio(column)
field ← CreateFieldSchema(column, inferred_type, confidence)
schema.AddField(field)
end for
reference_sig ← GenerateSchemaSignature(schema)
compatible, incompatible ← ValidateAllFiles(files, reference_sig)
overall_conf ← 0.7 × AvgFieldConf + 0.3 × CompatibilityRatio
return SchemaDetectionResult(schema, compatible, incompatible, overall_conf)
```

Temporal patterns are detected using a hierarchy of regular expressions: ISO format (YYYY-MM-DD), US format (MM/DD/YYYY), EU format (DD-MM-YYYY), and full ISO datetimes (YYYY-MM-DDTHH:MM:SS). Each detected field produces a FieldSchema object containing: name, detected_type, nullable, confidence, sample_values, unique_count, null_count, total_count, min_value, and max_value.

3.3.2 File-Based Ingestion (CSV, JSON, Parquet)

For file uploads, users place their data files in a directory mounted into the backend container. The ingestion process runs three steps: (1) file discovery by recursively scanning for files matching the selected format; (2) schema validation, where all files are checked against the reference schema and any with missing or extra columns are flagged with specific mismatch reasons; and (3) Bronze ingestion, where Apache Spark reads all compatible files, unions them with unionByName(allowMissingColumns=True), appends metadata columns, and writes to MinIO.

Algorithm 2: Dynamic Reader Selection

```
Input: spark, file_path, file_format, options
Output: DataFrame
switch file_format do
case "csv":
```

```
df ← spark.read
.option("header", options.header)
.option("sep", options.delimiter)
.option("inferSchema", true)
.csv(file_path)
case "json":
df ← spark.read.json(file_path)
case "parquet":
df ← spark.read.parquet(file_path)
end switch
df ← AddMetadataColumns(df, ingestion_timestamp, pipeline_id)
return df
```

3.3.3 REST API Ingestion

For REST API sources, users configure the endpoint URL, HTTP method, authentication type (none, basic, or bearer token), and an optional data_key using dot-notation to navigate nested JSON responses. For example, query.recentchanges extracts the recentchanges array nested inside a query object. The system also supports cursor-based pagination, enabling complete data extraction across paginated endpoints by following a configurable next_key in each response.

3.3.4 Apache Kafka Streaming Ingestion

For real-time streaming, the system integrates with Apache Kafka using offset-based consumption with exactly-once semantics. Schema detection consumes a configurable sample of messages (default: 50) using a temporary consumer group without committing offsets, so no data is skipped. The generated Bronze DAG then processes messages incrementally: it consumes with enable_auto_commit=False, writes data to MinIO as Parquet partitions, and commits offsets only after a successful write. This ensures that a failed run never silently loses data. Kafka metadata columns are automatically appended: _kafka_topic, _kafka_partition, _kafka_offset, and _kafka_timestamp.

3.4 Component 2: Agentic Data Transformation

The transformation layer is where AutoPipeline's AI capabilities are most visible. Users describe their desired transformations in plain English "convert temperature from Fahrenheit to Celsius and flag anomalies" and the system generates, validates, and tests the corresponding PySpark code before applying it to any real data. This conversational interface handles both Silver (cleaning) and Gold (enrichment) transformations.

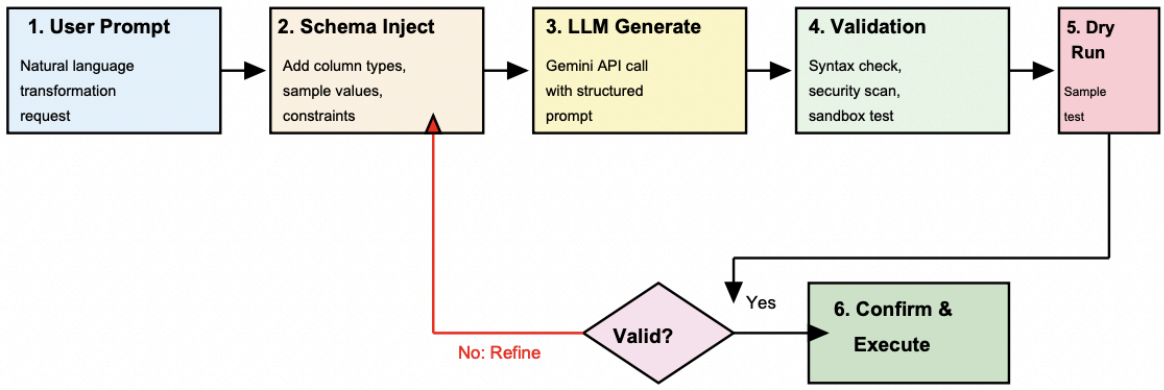

Figure 3: *LLM-powered code generation pipeline showing the flow from user prompt through schema injection, LLM generation, validation, dry-run testing, and confirmation with feedback loop for refinement.*

3.4.1 LLM Integration Architecture

The AI service communicates with the Google Gemini API using a carefully crafted system prompt that establishes role, capabilities, and output requirements. Three elements make this prompting strategy effective:

• Schema Context Injection: Before every API call, the current schema is formatted as a Markdown table showing column names, types, nullability, and sample values. This grounds the LLM in the actual data structure rather than having it guess column names.

• Rule Enforcement: The prompt explicitly specifies output format rules: ask for clarification (prefixed with[CLARIFICATION]) when requirements are ambiguous; wrap all generated code in a Python fence with exactly one function signature def transform(df, spark):; never write output (df.write).

• Conversation History Management: The full multi-turn conversation is included in every API call,enabling the LLM to refine previously generated code based on validation errors or user feedback without losing context.

The Gemini API is called with a low temperature (0.3) to prioritise code accuracy over creative variation, with a topP of 0.95 and a maximum of 4096 output tokens.

3.4.2 Code Generation Pipeline

Figure 2 illustrates the five-stage code generation pipeline. When a user submits a transformation request, the system executes the following algorithm:

Algorithm 3: LLM Code Generation Pipeline

Input: user_prompt, schema, conversation_history
Output: validated_code OR clarification_request

```
schema_context ← FormatSchemaAsMarkdown(schema)
messages ← BuildConversationMessages(
system_prompt, schema_context, history)
messages.Append(user_prompt)
payload ← {
contents: messages,
temperature: 0.3, // Low for code accuracy
topP: 0.95,
maxOutputTokens: 4096
}
response ← CallGeminiAPI(payload)
if ContainsClarificationPrefix(response) then
return ExtractClarificationRequest(response)
end if
code ← ExtractCodeFromFence(response)
valid, error ← StaticValidation(code) // Algorithm 4
if not valid then
return ValidationError(error)
end if
return validated_code
```

Algorithm 4: Static Code Validation

```
Input: code (Python string)
Output: (is_valid: bool, error_message: str)
// Enforce required function signature
if "def transform(" not in code then
return (false, "Missing transform(df, spark) signature")
end if
// Block dangerous operations
forbidden ← {"df.write", "os.system", "subprocess", "eval", "exec"}
for each op in forbidden do
if op in code then
return (false, "Forbidden operation: " + op)
end if
end for
// Python syntax check
try
Compile(code, "", "exec")
catch SyntaxError as e
return (false, "Syntax error at line " + e.lineno)
end try
return (true, "Validation passed")
```

3.4.3 Sandboxed Dry-Run Validation

Before any transformation is confirmed and applied to the full dataset, users can execute a dry-run on a sample of 10 rows. This provides a fast (<1 second) safety check that catches runtime errors the static validator cannot detect for example, referencing a column name that doesn't exist, or applying a string function to a numeric column.

The sandbox provides a restricted exec() environment. Dangerous built-ins such as eval, exec, compile, __import__, and open are blocked. Imports are restricted to a whitelist: pyspark, math, datetime, decimal, json, re, collections, functools, and itertools. A minimal local Spark session (512 MB driver memory, shuffle partitions = 2, UI disabled) is created to execute the function. If the dry-run fails, the error message is automatically fed back into the conversation context, and the LLM generates corrected code in the next turn.

3.5 Component 3: Pipeline Orchestration and DAG Generation

The orchestration layer generates Apache Airflow DAG files from project configuration using Jinja2 template rendering. This is the most critical reliability boundary in the system: while LLM-generated transformation code can contain bugs that the dry-run will catch, a malformed DAG file will fail silently at import time, making debugging extremely difficult. By generating DAGs from deterministic templates, AutoPipeline guarantees that every generated DAG file is syntactically correct and immediately executable in Airflow. This design directly implements the key finding from Prompt2DAG [13]: use LLMs for the parts of the pipeline where flexibility matters (transformation logic) and use templates for the parts where correctness is non-negotiable (orchestration structure).

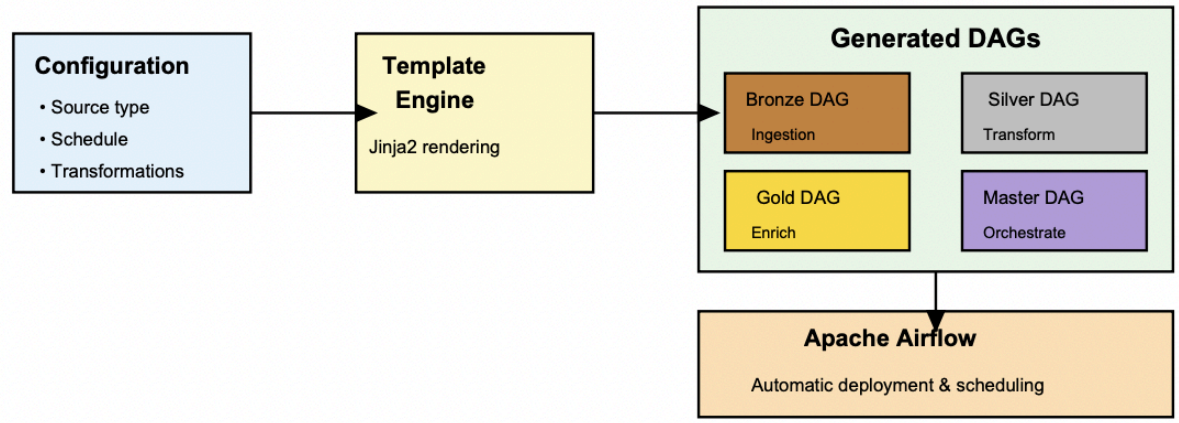

Figure 4: *Template-based DAG generation workflow showing configuration input, template engine processing, and automatic deployment to Apache Airflow.*

3.5.1 Four-DAG Generation Strategy

For each project, the system generates four interconnected DAGs:

• Bronze DAG (source-specific): Three templates handle different source types—file ingestion, REST API, and Kafka streams. Each template embeds the source configuration directly as Python literals.

• Silver DAG: Calls the backend API endpoint /api/silver/{id}/upload-to-silver which reads Bronze data,applies all confirmed Silver transformations sequentially, and writes output to the Silver bucket.

• Gold DAG: Calls /api/gold/{id}/upload-to-gold which reads Silver data, applies Gold transformations,writes to the Gold bucket, and optionally pushes to a PostgreSQL table.

• Master DAG: Chains Bronze → Silver → Gold using Airflow's TriggerDagRunOperator, providing a single-click end-to-end pipeline execution with proper dependency management.

The backend-API-call pattern for Silver and Gold DAGs provides an important architectural benefit:transformations can be updated and re-confirmed without regenerating the DAG files. The DAG simply calls the backend, which always executes the latest confirmed transformation code. All execution metrics are logged to PostgreSQL for auditability.

Algorithm 5: Template-Based DAG Generation

```
Input: project_config, transformations
Output: dag_files[]
dag_files ← []
// Select Bronze template based on source type
switch project_config.source_type do
case "file": template ← BRONZE_FILE_TEMPLATE
case "api": template ← BRONZE_API_TEMPLATE
case "kafka": template ← BRONZE_KAFKA_TEMPLATE
end switch
bronze_dag ← RenderTemplate(template, project_config)
dag_files.Append(bronze_dag)
if transformations.silver.Count > 0 then
silver_dag ← RenderTemplate(SILVER_TEMPLATE, project_config)
dag_files.Append(silver_dag)
end if
if transformations.gold.Count > 0 then
gold_dag ← RenderTemplate(GOLD_TEMPLATE, project_config)
dag_files.Append(gold_dag)
end if
master_dag ← RenderTemplate(MASTER_TEMPLATE, dag_files)
```

```
dag_files.Append(master_dag)
for each dag in dag_files do
WriteToAirflowDagsDir(dag)
end for
return dag_files
```

4. Case Studies

We conducted three end-to-end experiments to validate AutoPipeline across different ingestion patterns, transformation complexities, and data volumes. All experiments ran on a development machine with Docker Compose deploying the full 13-service stack. The Spark cluster used a single worker node (2 cores, 2 GB memory). Google Gemini 2.5-flash served as the LLM.

4.1 Case Study 1: CSV File Upload Flight Delay Dataset

Dataset: 50,000 flight records with attributes FlightNo, Airline, Origin, Destination, ScheduledDeparture,ActualDeparture, ScheduledArrival, ActualArrival, DelayMinutes, and Status.
Schema Detection: The algorithm identified 10 columns (5 strings, 3 timestamps, 1 integer, 1 categorical)with an overall confidence of 0.92 in 1.2 seconds. Bronze ingestion completed in 8.3 seconds.
Silver Transformation (natural language prompt): "Filter out cancelled flights, calculate the actual delay in minutes as the difference between ActualArrival and ScheduledArrival, and add a delay_category column that is 'On Time' if delay is less than 0, 'Minor' if 0–30, 'Moderate' if 30–60, and 'Severe' if more than 60 minutes."

The LLM generated correct PySpark code using F.unix_timestamp() for time difference and F.when().when().otherwise() for categorisation. Dry-run validated on 10 sample rows in 0.8 seconds. The Silver upload processed 45,230 records (4,770 cancelled flights filtered out) in 12.4 seconds.

Gold Transformation: After one clarification question about reference table location, the LLM generated a join-based aggregation producing 48 aggregated records in 9.8 seconds. The Gold data was exported to a PostgreSQL table in 0.8 seconds

4.2 Case Study 2: Apache Kafka Stream IoT Sensor Data

Data Source: A simulated IoT sensor stream producing JSON messages at ~100 messages/second to topic sensor-readings. Each message contained: sensor_id, temperature, humidity, pressure, timestamp, location.

Schema Detection: The system consumed 50 sample messages using a temporary consumer group(without committing offsets), detecting 6 fields (2 strings, 3 floats, 1 timestamp) with confidence 0.88 in 3.2 seconds.

Silver Transformation: "Convert temperature from Fahrenheit to Celsius, flag anomalies where temperature is below -20°C or above 50°C, and extract the hour from timestamp." The LLM correctly used (F - 32) * 5/9 and F.when() for anomaly flagging.

Gold Transformation: "Calculate hourly statistics per sensor: min, max, mean temperature and humidity.Flag hours with more than 3 anomalies." The aggregation used groupBy("sensor_id", "hour").agg() with multiple aggregate functions.

Incremental Processing: After 5 DAG runs over 2 hours, the Bronze layer contained 48,000 records across 5 Parquet partitions with zero duplicates, validating the exactly-once offset-commit semantics.

4.3 Case Study 3: REST API Source Wikipedia Recent Changes

Data Source: The Wikimedia API endpoint for recent changes, with nested JSON extraction using data key query.recentchanges. No authentication required.

Schema Detection: 500 records returned per request. The system detected 15 fields with confidence 0.95 the highest across all case studies in 1.8 seconds.

Silver Transformation: "Filter to only namespace 0 (main articles), exclude bot edits (users ending in 'bot' case-insensitively), parse the timestamp to a proper datetime, and calculate the revision size change." The LLM correctly used F.lower() with string matching and F.to_timestamp().

Gold Transformation: "Group by hour and user, count edits per user per hour, and create a summary with the top 10 editors per hour." Window functions were correctly used for ranking within each hour partition.

5. Results and Analysis

5.1 Performance Metrics

Table 3 summarises the timing and quality metrics across all three case studies.

Metric	CSV Upload	Kafka Stream	REST API
Schema Detection Time	1.2 s	3.2 s	1.8 s
Detection Confidence	0.92	0.88	0.95
Bronze Ingestion Time	8.3 s	4.1 s	2.1 s
Records Ingested	50,000	10,000	500
Silver Transform Code Gen	2.8 s	3.1 s	2.9 s
Silver Dry-Run Time	0.8 s	0.6 s	0.4 s
Silver Execution Time	12.4 s	8.7 s	1.2 s
Gold Transform Code Gen	4.2 s*	3.5 s	3.8 s
Gold Execution Time	9.8 s	6.2 s	0.9 s
PostgreSQL Push Time	0.8 s	0.5 s	0.2 s
Total Pipeline Time	**~40 s**	**~30 s**	**~12 s**

*Table 3: Performance metrics across all case studies. * Includes 1 clarification round.*

5.2 Development Effort Comparison

To quantify AutoPipeline's value proposition, Table 4 compares the estimated time required to build an equivalent pipeline manually (experienced data engineer) versus using AutoPipeline.

Task	Manual Approach	AutoPipeline	Reduction
Analyse CSV/analyse schema	~30 min	1.2 sec	99.9%
Write type mapping code	~20 min	0 sec	100%
Handle nulls / edge cases	~60 min	0 sec	100%
Write PySpark transformation code	~45 min	2.8 sec	99.9%
Debug and unit-test code	~30 min	0.8 sec	99.9%
Write Airflow DAG (boilerplate)	~60 min	0 sec	100%
Configure Airflow operators	~30 min	0 sec	100%
Set up MinIO / Spark / Airflow	~180 min	0 sec	100%
Total Estimated Time	**~8–10 hours**	**~2 minutes**	**99.6%**

Table 4: Development effort comparison — manual versus AutoPipeline

5.3 Skill Requirements Comparison

Beyond time savings, AutoPipeline eliminates the requirement for multiple deep technical skills. Table 5 shows that the only skill a user needs is domain knowledge of their data. All programming skills are replaced by the ability to describe requirements in natural language.

Skill	Manual Approach	AutoPipeline
Python programming	Required (advanced)	Not required
PySpark knowledge	Required (intermediate)	Not required
Apache Airflow expertise	Required (intermediate)	Not required
SQL / data modelling	Required	Not required
Docker / infrastructure	Required	Not required
Domain knowledge	Required	**Required**
Natural language description	N/A	**Required**

Table 5: Skill requirements comparison

5.4 Quality Observations

Schema Detection Accuracy: The heuristic type inference achieved confidence scores of 0.88–0.95 across all experiments. The weighted formula (0.7 × average field confidence + 0.3 × file compatibility ratio) correctly assigned lower scores to datasets with mixed types or unusual formats, providing a reliable quality signal to users.

LLM Code Generation Quality: Generated PySpark code was syntactically correct in all 10 test scenarios (5Silver + 5 Gold across the three case studies). The LLM appropriately requested clarification in the one ambiguous case (reference table location in Case Study 1) rather than making an incorrect assumption.Generated code was consistently clean, with meaningful docstrings and inline comments.

Dry-Run Effectiveness: During development, the sandboxed dry-run caught two issues: a column nametypo and an incorrect import statement. Both were resolved in a single LLM follow-up turn. The sub-second execution time on 10 sample rows enables rapid iteration with minimal user friction.

Kafka Offset Management: Exactly-once semantics worked correctly across 5 incremental DAG runs over 2 hours. Zero duplicate records were observed in the Bronze layer, validating the commit-after-write pattern.

DAG Generation Reliability: All generated DAG files were syntactically correct and immediately deployable in Airflow without manual editing. The backend-API-call architecture for Silver and Gold DAGs successfully decoupled transformation updates from DAG regeneration.

6. Discussion

6.1 Future Work

• Local LLM Deployment: Integrate Code Llama or similar open-weights models for fully offline operation, removing the API key and internet connectivity requirements.
• Schema Evolution Handling: Implement automatic detection of schema drift across Bronze versions and guided migration tooling.
• Multi-LLM Support: Add support for alternative LLMs (Claude, GPT-4, Llama) with automatic fallback when the primary model is unavailable.
• Advanced Transformation Patterns: Support for slowly changing dimensions (SCD Type 2), incremental aggregations, and native Spark Structured Streaming transformations.
• Cloud Deployment: Provide Terraform/Helm charts for production deployment on AWS, GCP, or Azure with managed Spark clusters and cloud-native object storage.

Please include following in your article:

For each of your 3 case studies, run the pipeline **10 times** and report mean ± standard deviation. (write simple script to repeat no need of manual)
Collect for all metrics

Run Case Study 1 (CSV) at 4 different data sizes: 10K, 50K, 100K, and 500K records. Plot ingestion time vs. record count. Plan similarly for other case studies

Based on the literature or github sources do the baseline comparison and fill the table

operations	Our model	Regular aiflow	Other product	Other product	Other product

7. Conclusion

We have presented AutoPipeline, a configuration-driven framework that transforms natural language descriptions into fully executable, production-grade data pipelines. By combining the flexibility of LLM-generated transformation code with the reliability of template-based DAG generation, AutoPipeline achieves what neither approach could deliver alone: a system that is simultaneously accessible to non-engineers and robust enough for production use. The framework's four key contributions work in concert: the intelligent schema detection engine eliminates the need for manual data profiling; the conversational transformation interface translates intent into validated PySpark code; the Medallion Architecture provides clear data lineage and supports incremental reprocessing; and the template-based DAG generator guarantees structurally correct Airflow orchestration without any manual configuration. Our three case studies covering CSV file uploads, Apache Kafka IoT streams, and REST API ingestion demonstrate the practical viability of this approach across diverse real-world scenarios. The quantitative results are compelling: schema detection confidence of 0.88-0.95, syntactically correct code generation in every test scenario, and a 99.6% reduction in end-to-end development time (from 8-10 hours to approximately 2 minutes). Perhaps most importantly, AutoPipeline removes the programming skill barrier entirely. A domain expert who understands their data can now build production pipelines without writing a single line of Python, PySpark, or Airflow configuration code. This democratisation of data engineering is the framework's most significant contribution.

References

[1] Apache Software Foundation. Apache Airflow. https://airflow.apache.org/, 2024.
[2] Apache Software Foundation. Apache NiFi. https://nifi.apache.org/, 2024.
[3] Prefect Technologies. Prefect: Modern Workflow Orchestration. https://www.prefect.io/, 2024.
[4] Dagster Labs. Dagster: Cloud-Native Orchestration. https://dagster.io/, 2024.
[5] dbt Labs. dbt (data build tool). https://www.getdbt.com/, 2024.
[6] Spotify. Luigi: Build Complex Pipelines of Batch Jobs. https://github.com/spotify/luigi, 2024.
[7] Prefect. Prefect Documentation. https://docs.prefect.io/, 2024.
[8] AWS. AWS Step Functions. https://aws.amazon.com/step-functions/, 2024.
[9] M. Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.

[10] B. Roziere et al. Code Llama: Open Foundation Models for Code. arXiv:2308.12950, 2023.
[11] A. Khot et al. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. ICLR, 2023.
[12] R. Li et al. StarCoder: May the Source Be with You! arXiv:2305.06161, 2023.
[13] A. Alidu, M. Ciavotta, F. De Paoli. Prompt2DAG: A Modular Methodology for LLM-Based Data Enrichment Pipeline Generation. arXiv:2509.13487, 2025.
[14] Databricks. Medallion Architecture. https://www.databricks.com/glossary/medallion-architecture, 2024.

Appendix A: System Deployment

The complete AutoPipeline system can be deployed on any machine with Docker and Docker Compose installed using the following four commands:

```
git clone https://github.com/sairam030/autonomus_datapipeline.git
cd autonomus_datapipeline
cp .env.example .env # then set GEMINI_API_KEY in .env
docker compose up -d --build
```

Once deployed, the following service endpoints are available:

Service	URL	Default Credentials
React Frontend UI	http://localhost:3001	—
FastAPI Backend (Swagger)	http://localhost:8000/docs	—
Apache Airflow UI	http://localhost:8085	admin / admin
MinIO Console	http://localhost:9011	minioadmin / minioadmin
Spark Master UI	http://localhost:8090	—

Table A1: Service endpoints after deployment

Appendix B: Database Schema Overview

PostgreSQL serves as the central metadata store. The 20 database tables are organised into logical groups covering pipeline lifecycle management, schema versioning, transformation tracking, execution logging, and audit trails.

Table	Purpose
`pipelines`	Core project definitions with status tracking
`data_sources`	Connection details for CSV, API, Kafka, DB sources
`schema_registry`	Versioned schema definitions with detection metadata
`bronze_ingestions`	Tracks each ingestion run with timing metrics and record counts
`silver_transformations`	AI-generated PySpark code with status (draft/confirmed/failed)
`gold_transformations`	Gold layer transformation code and configuration
`conversation_messages`	Complete LLM chat history per transformation task
`silver_executions`	Execution metrics for Silver uploads (records in/out, duration)
`gold_executions`	Execution metrics for Gold uploads
`postgres_pushes`	Records of Gold-to-PostgreSQL export operations
`audit_log`	Entity-level change tracking for all pipeline modifications

Table B1: Key PostgreSQL tables in the AutoPipeline metadata store